PromptLab

Prompt Engineering Toolkit for Claude, Bedrock & Vertex AI

Claude API · Amazon Bedrock · Vertex AI · React · FastAPI

Rahul Sangral
Data Analyst · AI Developer · Founder, Clavix Technologies
rahul.rishusangral@gmail.com · India · github.com/sangralrahul

Table of Contents

01. Executive Summary
02. Why Prompt Engineering Tooling?
03. Product Vision & Philosophy
04. System Architecture
05. Supported AI Providers
06. Prompt Version Control
07. A/B Testing Engine
08. Token Cost Tracker
09. Evaluation & Scoring System
10. React Frontend Design
11. FastAPI Backend
12. Prompt Template System
13. Experiment Management
14. Team Collaboration Features
15. Security & API Key Management
16. Performance Considerations
17. Testing & Quality
18. Deployment Architecture
19. Roadmap
20. Conclusion & Appendix

01. Executive Summary

PromptLab is an open-source developer toolkit for systematically engineering, testing, and optimising LLM prompts across multiple AI providers — Claude (Anthropic), Amazon Bedrock, and Google Vertex AI. It brings software engineering discipline to prompt development: version control, automated testing, cost tracking, and A/B comparison — all in a single React + FastAPI application.

- Version-control prompts like code with full diff history
- A/B test prompt variants across models side-by-side
- Track token usage and costs in real time across providers
- 3 AI providers supported in v1.0
- Export winning prompts as code snippets or API configs
- MIT-licenced, self-hosted

02. Why Prompt Engineering Tooling?

As LLMs become production infrastructure, prompt engineering has become a core engineering discipline. Yet most teams manage prompts in ad-hoc ways:

- Prompts stored in code comments, Notion pages, or Slack messages
- No version history — impossible to roll back a prompt that degraded performance
- No systematic A/B testing — teams guess which variant performs better
- No cost visibility — teams discover expensive prompts only on the billing statement
- Model comparisons done manually by copy-pasting between ChatGPT, Claude, and Gemini interfaces

PromptLab addresses all of these with a structured, developer-friendly workflow that integrates into existing CI/CD pipelines.

03. Product Vision & Philosophy

Core Principles

- Prompts are code: they deserve version control, review processes, and regression testing
- Empiricism over intuition: measure everything, decide by data
- Provider-agnostic: the best model for each task should be a data-driven choice, not a default
- Developer-first: CLI and API access as first-class citizens alongside the UI
- Privacy: prompts and outputs are never sent to external services beyond the configured LLM providers

04. System Architecture

PromptLab follows a classic three-tier architecture:

Frontend

React 18 + TypeScript + Vite. State management via Zustand. UI components built with Radix UI primitives and Tailwind CSS. Monaco Editor for prompt editing with LLM-specific syntax highlighting.

Backend

FastAPI (Python 3.11) serves as the API layer. Handles provider abstraction, experiment orchestration, token counting, and cost calculations.

Storage

PostgreSQL for prompt versions, experiments, and results. Redis for caching provider responses during A/B tests (avoids repeat API calls for identical inputs).

05. Supported AI Providers

Anthropic Claude (Direct API)

- claude-3-haiku-20240307, claude-3-sonnet-20240229, claude-3-opus-20240229
- claude-3-5-sonnet-20241022 supported
- System prompt, human turn, and assistant pre-fill all configurable
- Streaming supported with token-level timing metrics

Amazon Bedrock

- Claude 3 models via Bedrock converse API
- Titan Text, Llama 3, Mistral models also supported via same interface
- IAM-based authentication — no API keys required on EC2 with instance roles
- Region selection (us-east-1, eu-west-1, ap-southeast-1)

Google Vertex AI

- Gemini 1.5 Flash, Gemini 1.5 Pro
- Service account JSON authentication
- Project ID and location configurable per experiment

06. Prompt Version Control

Prompts are stored as versioned records in PostgreSQL with full diff history:

- Each save creates a new version (immutable — old versions never deleted)
- Version metadata: author, timestamp, change description, performance metrics
- Side-by-side diff view using diff-match-patch
- One-click rollback to any previous version
- Branch model: experiments fork from a base prompt version
- Git export: dump all prompt versions as .txt files for Git-based backup
- Semantic versioning: major (breaking restructure), minor (significant change), patch (wording tweak)

07. A/B Testing Engine

The A/B testing engine is the core of PromptLab:

Experiment Setup

- Define up to 5 prompt variants in an experiment
- Select providers and models independently per variant
- Define test inputs: single input, CSV batch, or synthetic input set
- Set evaluation criteria: manual scoring, automated rubric, or regex match

Execution

- All variants run concurrently via asyncio
- Responses cached by input hash (1-hour TTL) to avoid duplicate API calls
- Progress tracked in real time via WebSocket
- Automatic retry on provider rate limit (exponential backoff, max 3 retries)

Results

- Side-by-side output comparison
- Win rate statistics with 95% confidence interval (binomial test)
- Per-variant: latency p50/p95, token count, cost, quality score
- Export results as CSV for external analysis

08. Token Cost Tracker

Real-time cost tracking across all providers:

- Input and output tokens counted per request using tiktoken and provider-reported usage
- Costs calculated using current provider pricing (updated monthly via config)
- Per-experiment cost summary: total, per-variant, per-input
- Cumulative spend dashboard: daily, weekly, monthly breakdowns
- Budget alerts: configurable spend limits with email/Slack notifications
- Cost projection: estimated monthly spend based on current usage trajectory
- Provider comparison: side-by-side cost-per-quality-point for informed model selection

09. Evaluation & Scoring System

Manual Scoring

Human evaluators rate outputs on configurable rubrics (1–5 scale per criterion). Scores averaged across evaluators with inter-rater reliability reported (Cohen's Kappa).

Automated Rubric

An LLM-as-judge pattern: a separate Claude call evaluates each output against the rubric criteria. Prompt template for the judge is versioned and configurable.

Exact Match

For classification or extraction tasks, outputs compared against a gold-standard answer set using exact match, fuzzy match (Levenshtein), or regex pattern matching.

Custom Metrics

Users can register custom Python evaluation functions via the plugin system. Example: ROUGE score for summarisation, BLEU for translation, custom regex for structured output validation.

10. React Frontend Design

The frontend prioritises developer ergonomics:

- Monaco Editor with custom LLM prompt language support (variable highlighting: {{variable_name}})
- Variable injection panel: define test variables and see prompt preview update in real time
- Split-pane layout: prompt editor left, output/comparison right
- Dark theme by default (matching the portfolio design system)
- Keyboard shortcuts for power users: Ctrl+Enter to run, Ctrl+S to save, Ctrl+D to diff
- Response streaming displayed token-by-token with timing markers
- Responsive layout: works on iPad for testing on the go

11. FastAPI Backend

The backend API is structured for extensibility:

- Provider abstraction layer: uniform interface across all AI providers
- Async throughout: provider calls, database queries, and cache operations all async
- Background tasks: experiment runs dispatched as background tasks with progress WebSocket
- Rate limit middleware: client-side rate limiting to prevent accidental API overspend
- OpenAPI docs at /docs — all endpoints fully documented
- Webhook support: POST to user-defined URL on experiment completion

12. Prompt Template System

PromptLab supports Jinja2-style variable injection in prompts:

- Variables defined as {{variable_name}} in prompt text
- Variable panel auto-detects all variables and prompts for values
- Default values configurable per variable
- Variable sets: save named sets of variable values for repeated testing
- Nested templates: prompts can include other prompt snippets via {% include %}
- Template library: shared prompt fragments for common patterns (chain-of-thought, few-shot examples, output format instructions)

13. Experiment Management

- Experiment lifecycle: Draft !’ Running !’ Complete !’ Archived
- Experiment cloning: duplicate a successful experiment to iterate further
- Tags and search: find experiments by tag, model, date range, or outcome
- Pinning: pin best-performing prompt versions as "production" references
- Changelog: auto-generated changelog from version metadata for team visibility
- Audit log: all experiment runs logged with user, timestamp, and configuration snapshot

14. Team Collaboration Features

- User accounts with role-based access (Admin, Editor, Viewer)
- Prompt collections: organise prompts into projects with team-level access control
- Comments: leave inline comments on prompt versions
- Review workflow: optional peer review required before promoting a prompt to "production"
- Activity feed: real-time feed of team experiment runs and prompt updates
- Export/import: share prompt libraries between teams as ZIP archives

15. Security & API Key Management

- Provider API keys stored encrypted in PostgreSQL (Fernet symmetric encryption, key from env var)
- Keys never returned to frontend — backend uses them directly
- Per-user key storage: each user manages their own provider credentials
- Organisation-level shared keys: admins can provision shared keys for team use
- Key rotation: update keys without downtime; pending experiments complete with old key
- Audit log: every API call logged with masked key ID for compliance tracing

16. Performance Considerations

- Provider calls run concurrently via asyncio.gather — 5-variant experiment same latency as 1
- Redis caching eliminates duplicate API calls for identical (prompt, model, input) triples
- WebSocket progress updates: avoid long-polling for experiment status
- Database connection pooling: asyncpg with max 20 connections
- Pagination: all list endpoints paginated (default 20 items, max 100)
- Frontend virtualisation: large experiment result lists rendered with react-window

17. Testing & Quality

- Unit tests: 54 tests covering provider adapters, cost calculations, and scoring algorithms
- Integration tests: real Claude Haiku API calls in integration suite (low-cost model)
- E2E tests: Playwright tests for critical user flows (create prompt, run experiment, view results)
- Provider mocking: httpx mock for all provider calls in unit/integration CI
- Type safety: TypeScript strict mode on frontend, mypy strict on backend
- Accessibility: axe-core automated accessibility testing on all React components

18. Deployment Architecture

- Docker Compose: app (FastAPI) + frontend (Vite dev server) + PostgreSQL + Redis
- Production: FastAPI behind Nginx, React served as static files from Nginx
- Environment: all secrets via environment variables (no .env files in production)
- Health checks: /health (app), /ready (app + DB + Redis)
- Logging: structured JSON to stdout, aggregated by CloudWatch Logs or Datadog
- Backup: pg_dump scheduled daily to S3 with 30-day retention

19. Roadmap

v1.1 (Q3 2026)

- OpenAI GPT-4o + o1 provider support
- CLI tool for prompt execution and experiment runs
- Prompt linting: detect common prompt anti-patterns
- Integration with LangSmith and Weights & Biases

v2.0 (Q4 2026)

- Visual prompt flow builder for chained LLM calls
- Automated prompt optimisation (DSPy-style)
- Multi-modal prompts (image + text)
- SaaS hosted version with team billing

20. Conclusion & Appendix

PromptLab brings engineering rigour to one of AI development's most undisciplined areas: prompt design. By treating prompts as versioned, testable, measurable artefacts, teams can iterate faster, reduce costs, and ship more reliable AI features.

The project demonstrates Rahul Sangral's ability to identify developer tooling gaps in the AI ecosystem and build production-quality solutions that solve real workflow problems.

Rahul Sangral
rahul.rishusangral@gmail.com · India

Appendix — Default Provider Pricing (April 2026)

- Claude 3 Haiku: $0.25 / 1M input tokens, $1.25 / 1M output tokens
- Claude 3.5 Sonnet: $3.00 / 1M input tokens, $15.00 / 1M output tokens
- Gemini 1.5 Flash: $0.075 / 1M input tokens, $0.30 / 1M output tokens
- Amazon Bedrock Titan Text: $0.0008 / 1K input tokens, $0.0016 / 1K output tokens

Appendix — Environment Variables

- ANTHROPIC_API_KEY
- AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY / AWS_REGION
- GOOGLE_APPLICATION_CREDENTIALS (path to service account JSON)
- DATABASE_URL (PostgreSQL)
- REDIS_URL
- ENCRYPTION_KEY (Fernet key for API key storage)
- JWT_SECRET_KEY